

Лекция 6

Память, защита памяти и прерывания в RISC-V-системе

Содержание, часть 1

1. От ядра к системе
2. Почему память тормозит процессор
3. Memory hierarchy
4. Cache memory
5. Cache line, tag/index/offset
6. Hit, miss и политики записи
7. Cache в FPGA
8. Virtual memory

Содержание, часть 2

- 9. Address translation и page tables
- 10. TLB и page fault
- 11. Memory protection
- 12. PMP
- 13. Exceptions и interrupts
- 14. Trap mechanism в RISC-V
- 15. Interrupt controller
- 16. Итоговая SoC-картина

1. В предыдущей лекции

В лекции 5 мы рассмотрели, как процессор исполняет больше инструкций за единицу времени:

- branch prediction;
- speculation;
- out-of-order execution;
- register renaming;
- multithreading и SMT.

Но быстрый конвейер бесполезен, если каждая инструкция ждёт медленную память или процессор не умеет безопасно работать с ОС и периферией.

Что добавляется на системном уровне

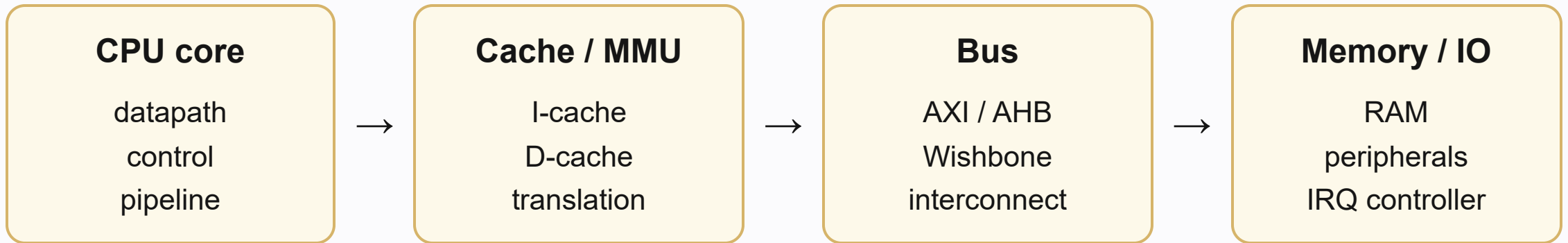
Память

- I-cache и D-cache
- RAM
- memory bus
- virtual memory
- address translation
- page tables
- memory protection

События и периферия

- memory-mapped peripherals
- exceptions
- interrupts
- interrupt controller
- timer interrupt
- external interrupt
- trap handler

RISC-V SoC



На системном уровне мы смотрим не только на ядро, но и на всю SoC: как CPU связан с памятью, периферией и внешними событиями.

4. Почему память — узкое место

Каждый такт процессору необходимо:

- брать новую инструкцию;
- читать операнды;
- иногда читать или писать данные в память.

Проблема:

ALU может быть быстрой, а внешняя память — на порядки медленнее.

Если каждый `lw` ждёт DDR, pipeline регулярно простаивает.

5. Уровни памяти

Чем ближе память к ядру, тем она быстрее, но тем меньше её объём.

Уровень	Где находится	Задержка
Registers	внутри ядра	~1 такт
L1 cache	рядом с pipeline	~1–4 такта
L2 cache	рядом с ядром / кластером	~8–20 тактов
BRAM / SRAM	внутри FPGA / SoC	~1–3 такта
DDR / DRAM	внешняя память	~50–300+ тактов
Flash / SD	хранилище	1000+ тактов

Числа примерные

6. Проблемы связанные с удаленностью памяти

Pipeline может выполнять инструкции каждый такт, но доступ к памяти может занимать десятки или сотни тактов.

Что хочет CPU	Что может произойти
Получить следующую инструкцию	instruction fetch ждёт память
Выполнить <code>lw</code>	данные ещё не пришли
Выполнить <code>sw</code>	запись ждёт bus / memory controller
Продолжить pipeline	стадия MEM блокирует следующие инструкции

Быстрый pipeline не спасает, если он постоянно ждёт медленную память.

7. Кэш. Локальность данных

Cache — маленькая быстрая память между ядром и основной памятью, которая хранит копии недавно использованных данных или инструкций

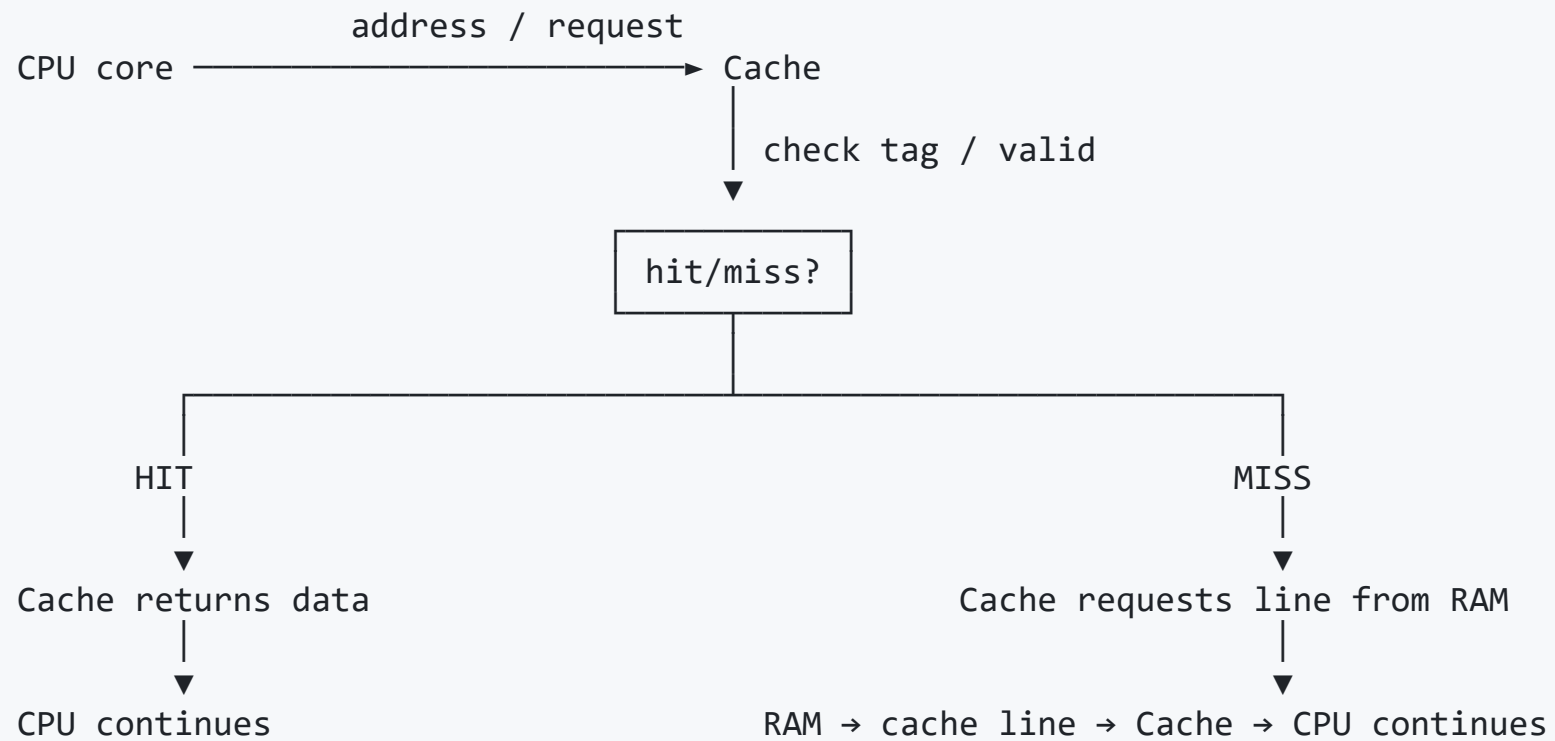
Кэш хорошо работает потому, что программы часто обращаются к памяти предсказуемо.

Вид локальности	Идея	Пример
Временная локальность	если данные использовались недавно, они могут понадобиться снова	переменная в цикле
Пространственная локальность	если использовался один адрес, скоро могут понадобиться соседние адреса	проход по массиву

Кэш использует локальность:

он хранит рядом с ядром то, что программа, скорее всего, скоро снова попросит.

8. Пример работы Кэш



9. Cache line

Данные обычно переносятся не по одному байту и не по одному слову, а блоками.

Cache line — минимальный блок данных, который cache загружает из памяти.

Пример:

Адреса в памяти:

0x1000 0x1004 0x1008 0x100C



одна cache line

Если программа читает массив подряд, это выгодно.

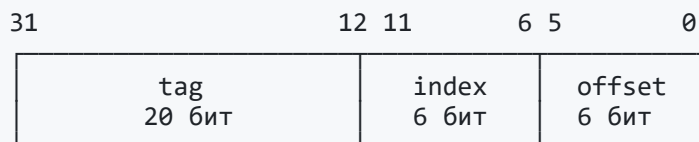
Разбиение адреса: tag / index / offset

Допустим:

- адрес 32-битный;
- cache = 4 KiB;
- cache line = 64 байта;
- количество строк cache = cache / cache line

Тогда:

Поле	Размер	Зачем нужно
offset	6 бит	выбрать байт внутри cache line
index	6 бит	выбрать строку cache
tag	20 бит	проверить, та ли область памяти лежит в строке



11. Кэш прямого отображения. Direct-mapped cache

В direct-mapped cache каждый адрес может попасть только в одну конкретную строку cache.

index = часть адреса

Адрес A → строка 0

Адрес B → строка 1

Адрес C → строка 2

Адрес D → строка 3

Адреса - которые могут вытеснять друг друга

0x8000_1040

0x8000_2040

0x8000_3040

0x8000_4040

Плюс: простая и быстрая схема.

Минус: разные адреса могут постоянно вытеснять друг друга.

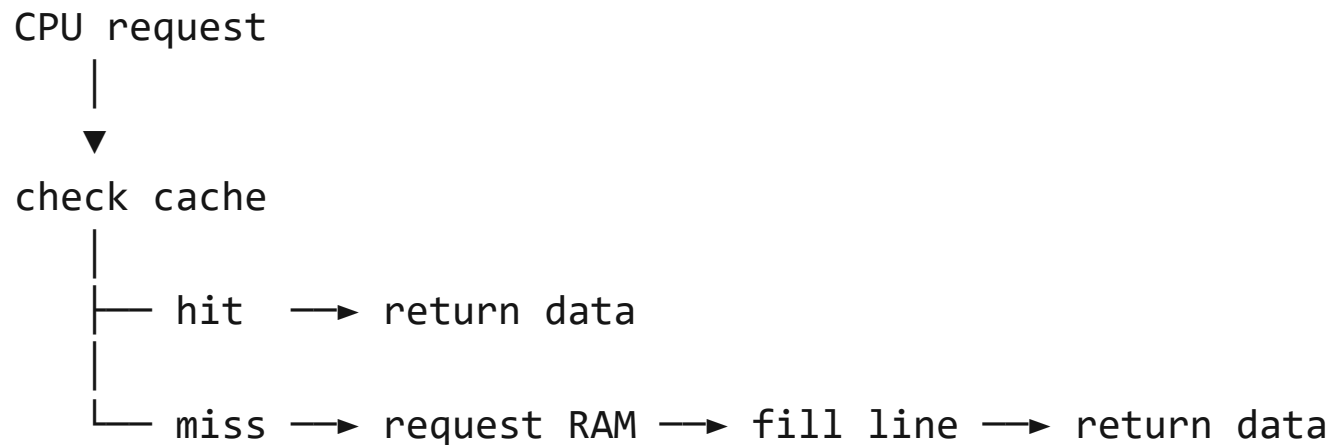
12. Cache hit и cache miss

Cache hit

Данные уже есть в cache.
CPU получает данные быстро.

Cache miss

Данных нет в cache.
Нужно идти в память, загрузить cache line и подождать.



13. Valid bit, dirty bit, tag

Обычно строка cache хранит не только данные.

Поле	Зачем нужно
Data	Сама cache line
Tag	Проверка, какие адреса лежат в этой строке
Valid bit	Есть ли в строке корректные данные
Dirty bit	Были ли данные изменены и ещё не записаны в RAM

14. Политики записи

Write-through

Запись идёт и в cache, и сразу в память.

Write-back

Запись сначала меняет только cache line.

В память данные уходят позже, при вытеснении.

В write-back cache нужен `dirty bit`, чтобы понимать, надо ли сохранять строку обратно в память.

15. Write-allocate и no-write-allocate

Что делать при записи, если адреса нет в cache?

Политика	Идея
Write-allocate	Сначала загрузить cache line, потом записать в неё
No-write-allocate	Не загружать строку в cache, а писать напрямую дальше

Обычно:

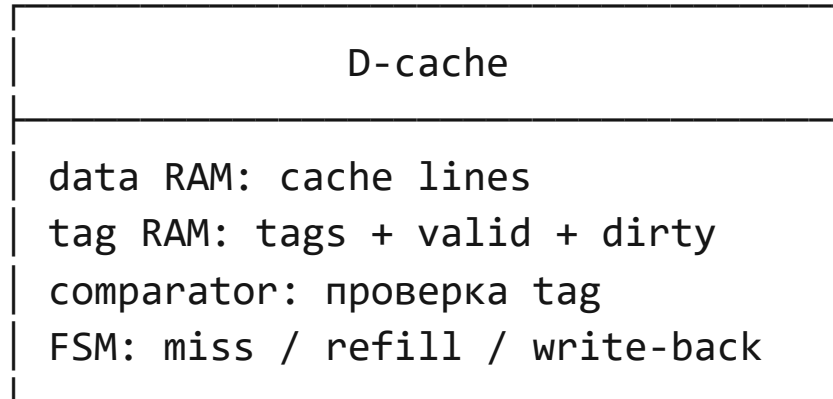
- write-back часто сочетается с write-allocate;
- write-through может сочетаться с no-write-allocate.

16. Пример: `lw` через cache

```
lw t0, 0(a0)
```

1. ALU считает адрес: $a0 + 0$
2. D-cache делит адрес на tag/index/offset
3. По index выбирается строка cache
4. Сравнивается tag
5. Если valid=1 и tag совпал → hit
6. Если нет → miss, запрос в RAM
7. После загрузки cache line нужное слово попадает в t0

17. Cache в FPGA: из чего он состоит



Обычно это BRAM/URAM + компаратор + контроллер состояний.

18. I-cache и D-cache

I-cache

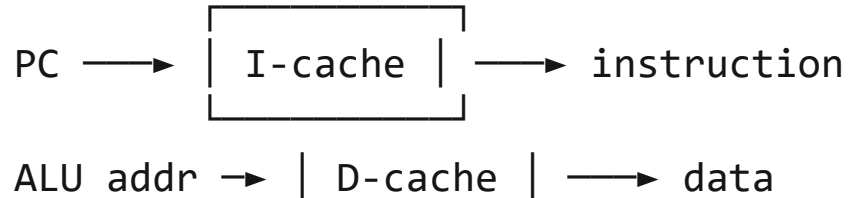
Хранит инструкции.

Помогает не тормозить instruction fetch.

D-cache

Хранит данные.

Помогает ускорить `lw`, `sw` и работу со стеком.



19. Почему доступ к памяти становится сложнее

Без кэша кажется, что каждое обращение CPU сразу идёт в RAM.

С кэшем это уже не так.

Что делает программа	Что может произойти на самом деле
<code>lw</code> читает адрес	данные могут прийти из D-cache, а не из RAM
<code>sw</code> пишет адрес	запись может остаться в cache и попасть в RAM позже
CPU читает инструкцию	инструкция может прийти из I-cache
устройство пишет в память через DMA	CPU может видеть старую копию в D-cache

Кэш ускоряет доступ к памяти, но при этом добавляются некоторые сложности, которые приходится учитывать

20. Virtual memory: зачем нужны виртуальные адреса

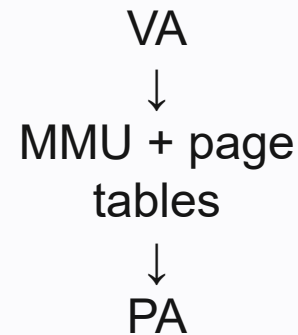
Virtual address space

Процесс А видит:

- code
- stack
- heap
- libraries

Процесс В видит:

- code
- stack
- heap
- libraries



Physical memory

Реальная RAM:

- страницы процесса А
- страницы процесса В
- страницы kernel
- memory-mapped devices

Виртуальные адреса дают каждому процессу своё адресное пространство, а MMU переводит эти адреса в реальные физические адреса RAM.

21. Physical address vs virtual address

Physical address

Реальный адрес на шине памяти.
Именно его видит RAM и контроллер памяти.

Virtual address

Адрес, который использует программа.
Перед доступом к памяти он должен быть переведён в physical address.

virtual address → MMU → physical address

22. Зачем нужна виртуальная память

Virtual memory нужна не только ради «большой памяти».

Возможность	Что даёт
Изоляция процессов	Один процесс не портит память другого
Защита ОС	User-код не пишет в kernel memory
Удобная загрузка программ	Каждая программа может иметь похожую карту памяти
Demand paging	Страницы можно подгружать по необходимости
Memory-mapped files	Файл может выглядеть как область памяти

23. Страницы памяти

Память делится на страницы фиксированного размера.

Типичный размер страницы: **4 KiB**.

Virtual address space

┌──────────┐ page 0

┌──────────┐ page 1

┌──────────┐ page 2

┌──────────┐ page 3

┌──────────┐ ...

Перевод идёт не для каждого байта отдельно, а для страницы.

24. Page table

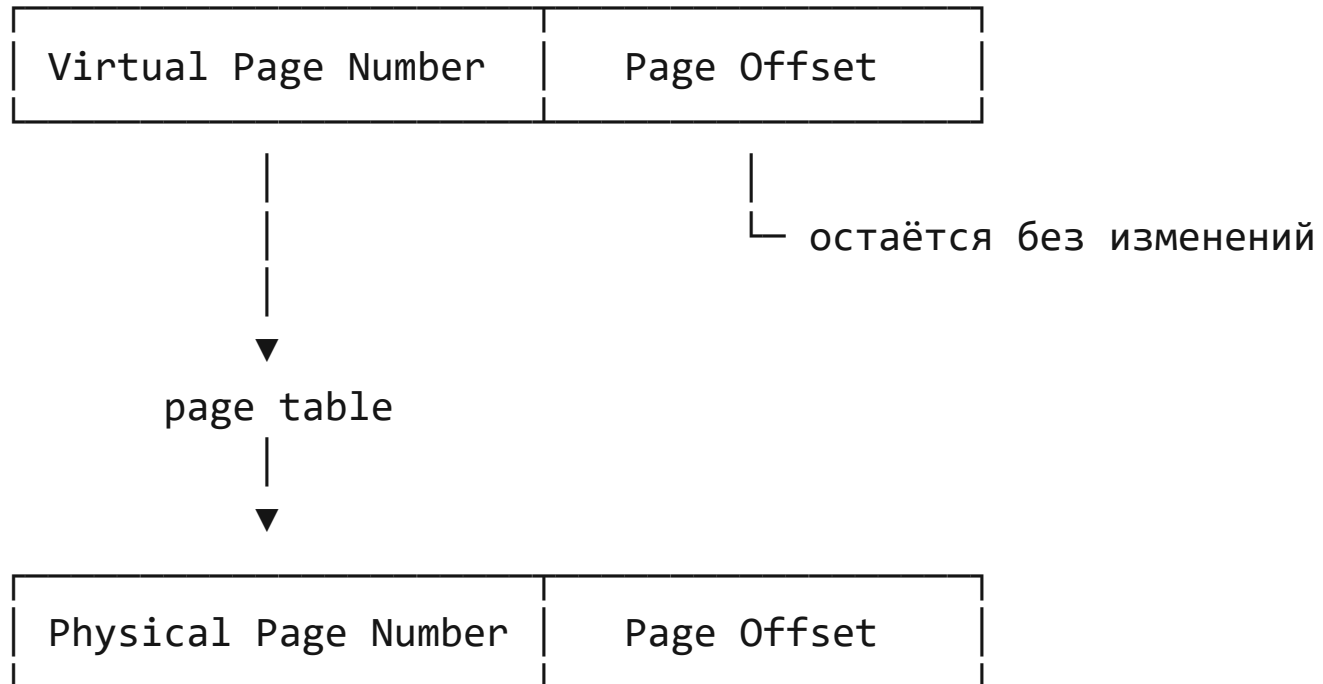
Page table — таблица, которая говорит, куда виртуальная страница отображается в физической памяти.

```
Virtual page 5 → Physical page 12  
Virtual page 6 → Physical page 21  
Virtual page 7 → not present  
Virtual page 8 → Physical page 3
```

Если страницы нет или доступ запрещён, возникает исключение.

25. Address translation

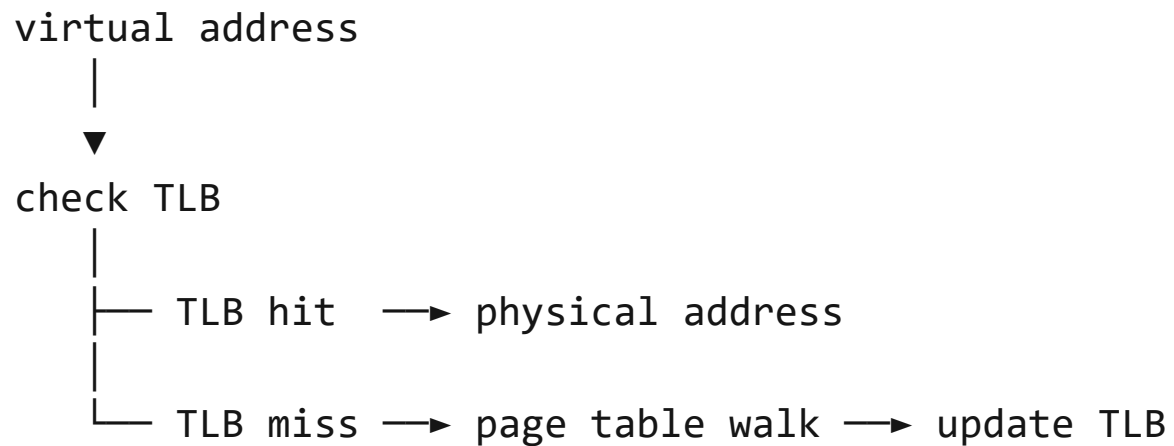
Виртуальный адрес делится на:



27. TLB

Доступ в page table для каждого доступа к памяти - дорогая операция

TLB — Translation Lookaside Buffer. Это cache для трансляций virtual page → physical page.



28. Page fault

Page fault возникает, когда адрес нельзя нормально перевести или использовать.

Причины:

- страница не присутствует в памяти;
- нет прав на чтение, запись или исполнение;
- обращение нарушает правила режима доступа;
- таблица страниц содержит некорректную запись.

Page fault — иногда ОС может обработать его и продолжить выполнение программы.

29. Bare-metal, RTOS и Linux

Система	Обычно нужна MMU?	Комментарий
Bare-metal	нет	Код сам управляет памятью
Маленькая RTOS	редко	Может хватить MPU/PMP
Linux	да	Нужна полноценная virtual memory

MPU/PMP - Memory Protection Unit/Physical Memory Protection

Многие простые FPGA RISC-V cores могут жить без MMU. Но для Linux-class системы MMU становится обязательной частью дизайна.

30. Memory protection: зачем запрещать доступ

Без защиты памяти любой код может:

- испортить память ядра ОС;
- записать в память другого процесса;
- выполнить данные как код;
- случайно обратиться к периферии;
- сломать систему из-за ошибки указателя.

Memory protection нужна не только от злоумышленника, но и от обычных багов.

31. Уровни привилегий в RISC-V

RISC-V разделяет уровни привилегий.

Режим	Назначение
M-mode	Machine mode, самый привилегированный уровень
S-mode	Supervisor mode, обычно ядро ОС
U-mode	User mode, обычные программы

32. Page permissions

Запись page table может содержать права доступа.

Бит	Смысл
R	страницу можно читать
W	в страницу можно писать
X	из страницы можно исполнять инструкции
U	доступна из user mode
V	запись валидна

33. PMP — Physical Memory Protection

PMP защищает физические области памяти.

Он может работать даже без виртуальной памяти.

Physical memory

0x0000_0000 — ROM: R X

|

0x2000_0000 — RAM: R W X

|

0x4000_0000 — MMIO: R W, no execute

PMP полезен для embedded и bare-metal систем.

34. MMU vs PMP

Механизм	Что делает	Где особенно полезен
MMU	Переводит virtual address в physical address	Linux, процессы, virtual memory
Page permissions	Защищают страницы виртуальной памяти	User/Supervisor разделение
PMP	Защищает физические регионы памяти	Bare-metal, RTOS, firmware

MMU отвечает за трансляцию адресов. PMP — за аппаратные границы доступа к physical memory.

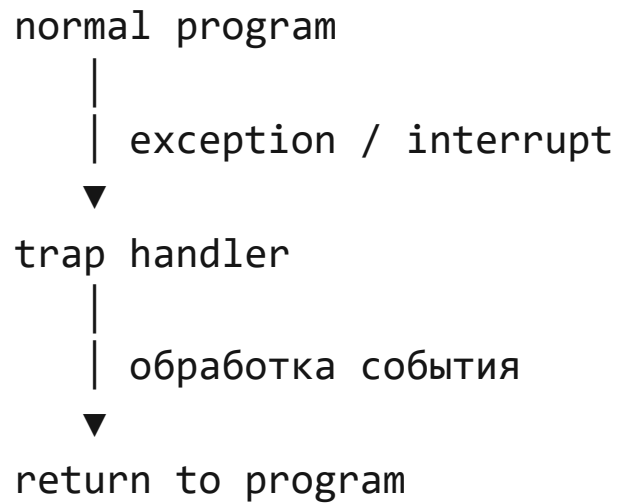
35. Exceptions и interrupts

Событие	Когда возникает	Пример
Exception	Из-за текущей инструкции	illegal instruction, load fault, page fault
Interrupt	Асинхронно, извне потока инструкций	timer, UART, GPIO, DMA

Оба типа событий в RISC-V попадают в общий механизм — **trap**.

36. Trap mechanism

Trap — переход процессора в специальный обработчик события.



Trap позволяет ОС или firmware перехватить ситуацию и принять решение.

37. Важные CSR для trap в M-mode

CSR	Зачем нужен
<code>mtvec</code>	адрес обработчика trap
<code>sepc</code>	адрес инструкции, куда нужно вернуться
<code>mcause</code>	причина trap
<code>mtval</code>	дополнительная информация: адрес, инструкция и т.п.
<code>mstatus</code>	флаги состояния и разрешения прерываний

Для S-mode существуют аналогичные supervisor CSR: `stvec`, `sepc`, `scause`, `stval`, `sstatus`.

38. Вход в обработчик

При trap процессор:

1. определяет причину события;
2. записывает причину в `mcause`;
3. сохраняет адрес возврата в `mepc`;
4. обновляет состояние в `mstatus`;
5. переходит по адресу из `mtvec`.

event → save cause/pc/status → jump to mtvec

39. Возврат через `mret`

Когда обработчик закончил работу, нужно вернуться назад.

```
mret
```

`mret` :

- возвращаемся в исходное состояние;
- восстанавливаем состояние interrupt enable;
- возвращает PC из `перс` .

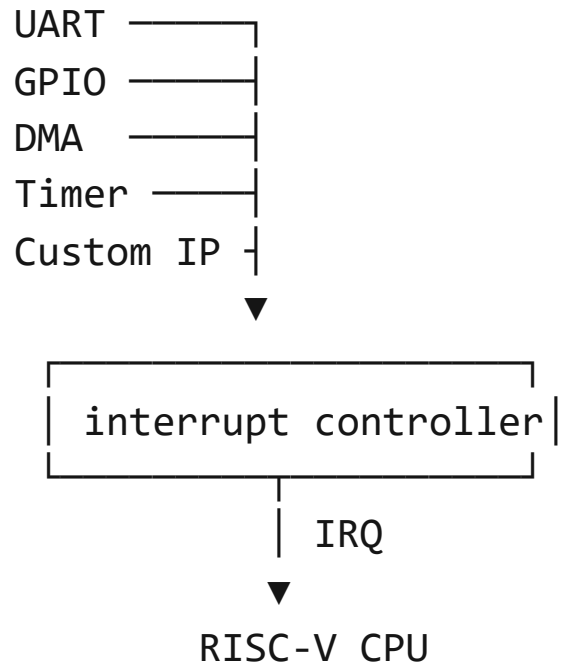
40. Зачем нужен interrupt controller

У системы может быть много источников прерываний:

- UART;
- GPIO;
- SPI/I2C;
- Ethernet;
- DMA;
- таймеры;
- внешние IP-блоки в FPGA.

Interrupt controller собирает запросы от устройств и помогает CPU понять, какое событие нужно обработать первым.

41. Interrupt controller как диспетчер



Контроллер может хранить pending-биты, enable-биты, приоритеты и номер активного IRQ.

42. Pending / enable / priority

Механизм	Смысл
Pending	событие уже произошло и ждёт обработки
Enable	разрешено ли это прерывание
Priority	насколько оно важное относительно других
Claim	CPU забирает номер IRQ в обработку
Complete	CPU сообщает, что обработка завершена

43. local interrupts и external interrupts

В RISC-V-платформах часто разделяют:

Local interrupts

Софтовые прерывания и прерывания таймера, близкие к ядру.

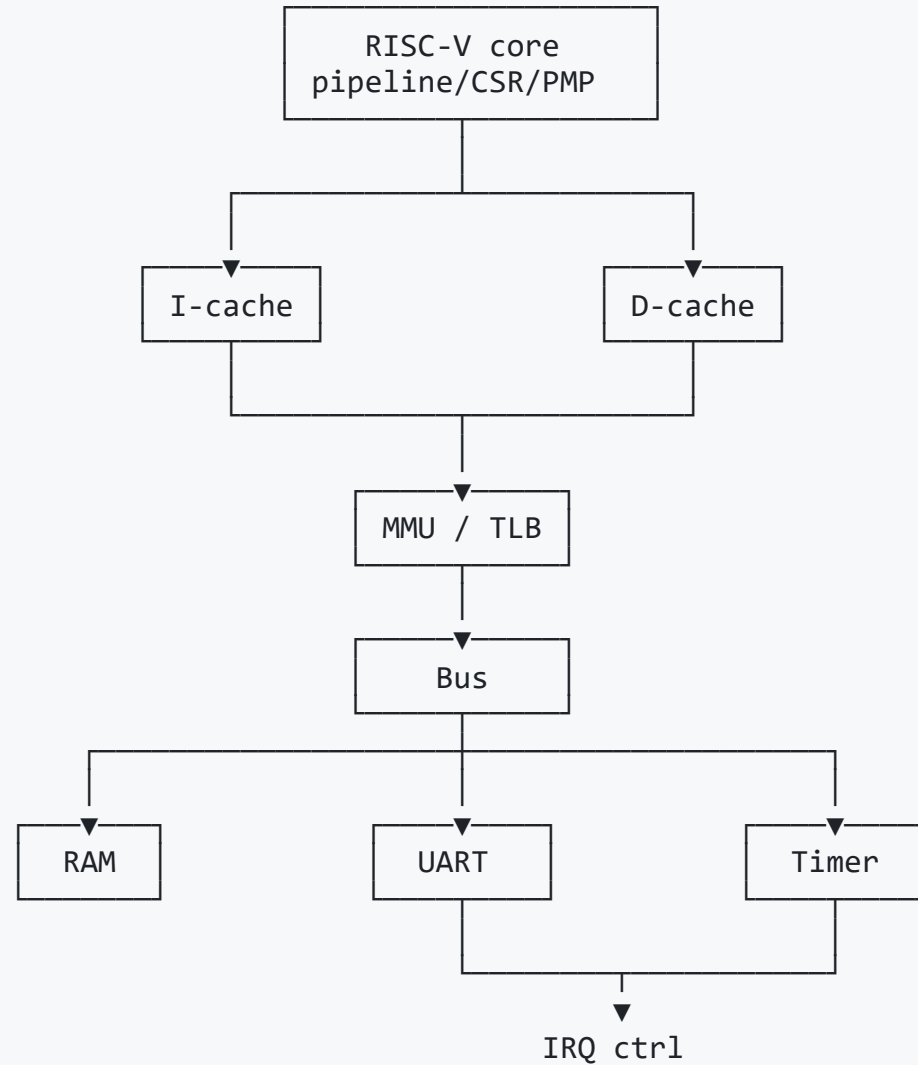
External interrupts

Прерывания от внешней периферии.

44. Пример: UART interrupt

1. UART получил байт
2. UART выставил interrupt request
3. Interrupt controller пометил IRQ как pending
4. CPU получил external interrupt
5. CPU вошёл в trap handler по mtvec
6. Handler прочитал причину события
7. Handler забрал байт из UART RX-регистра
8. Handler сообщил interrupt controller: complete
9. CPU вернулся через mret

45. Итоговая SoC-картина



47. Что мы прошли за курс

Уровень	Что изучали
ISA	инструкции, регистры, форматы команд
Datapath	ALU, registers, memory, immediate generation
Control	сигналы управления, выполнение инструкции
Pipeline	стадии, hazards, forwarding, stalls
Performance	prediction, speculation, OoO, multithreading
System	cache, MMU, protection, traps, interrupts

48. Что еще можно изучить и сделать

- написать простой direct-mapped cache;
- добавить memory-mapped UART и timer;
- сделать trap handler на bare-metal C/ASM;
- попробовать PMP-регионы;
- изучить AXI/AHB/Wishbone interconnect;
- посмотреть open-source RISC-V cores;
- отдельно изучить Linux-capable RISC-V SoC.